

Tutorial 3 – Listas enlazadas e iteradores

Ingeniería en Inteligencia Artificial — Algoritmos y Estructura de Datos

2C 2026 · Docente: Ing. Magali Marijuan

- Tutorial 3 – Listas enlazadas e iteradores
- Objetivos de la clase
- El nodo: la unidad básica
- Lista simplemente enlazada
 - Implementación de la lista simple
 - Insertar al inicio
 - Insertar al final
 - Buscar un elemento
 - Eliminar por valor
 - Recorrer la lista
 - Tamaño de la lista
 - Destructor: liberar todos los nodos
 - Repaso de complejidades (lista simple)
- Lista doblemente enlazada
 - Estructura del nodo doble
 - Insertar al final (lista doble)
 - Eliminar al final (lista doble)
 - Repaso de complejidades (lista doble)
- Lista circular
 - Estructura de la lista circular
 - Insertar al final (lista circular)
 - Recorrer la lista circular
 - Repaso de complejidades (lista circular)
- Iteradores
 - ¿Qué es un iterador?
 - Implementando un iterador propio
 - Cómo se traduce el for basado en rango
 - ¿Por qué desacoplar el recorrido de la estructura?
- Localidad espacial y cache
 - ¿Por qué importa la localidad espacial?
 - Vector vs lista enlazada: el impacto real
- Costo amortizado de `std::vector`
 - ¿Qué pasa cuando el vector se llena?
 - Análisis amortizado

- [Comparación con arreglo y `std::vector`](#)
- [Resumen general de complejidades](#)

Objetivos de la clase

- Entender qué es un nodo y cómo se encadenan para formar una lista.
- Implementar a mano una lista simplemente enlazada: insertar, borrar, buscar, recorrer y destruir.
- Extender la implementación a listas doblemente enlazadas y circulares.
- Comprender qué es un iterador y por qué es una abstracción que desacopla el recorrido de una estructura de su representación interna.
- Comparar las listas enlazadas con el arreglo dinámico (`std::vector`) en términos de complejidad y uso de memoria.

El nodo: la unidad básica

Una lista enlazada no es un bloque contiguo de memoria como un arreglo. Es una colección de **nodos** dispersos en el *heap*, donde cada nodo guarda un dato y **un puntero al siguiente nodo**.

```
struct Nodo {  
    int dato;  
    Nodo* siguiente;  
  
    Nodo(int valor, Nodo* sig = nullptr)  
        : dato(valor), siguiente(sig) {}  
};
```

Pregunta para pensar: si los nodos están dispersos en el heap, ¿por qué recorrer una lista enlazada suele ser más lento en la práctica que recorrer un `std::vector` del mismo tamaño, aunque ambos sean $O(n)$?

Cada nodo se reserva individualmente con `new` y debe liberarse individualmente con `delete`. Esta es la diferencia clave frente al arreglo: la lista **crece de a un nodo por vez**, sin necesidad de pedir un bloque más grande y copiar todo.

Lista simplemente enlazada

Invariante: cada nodo conoce al siguiente. El último nodo apunta a `nullptr`.

Implementación de la lista simple

```
class ListaSimple {
private:
    Nodo* primero;
    Nodo* ultimo;
    size_t largo;

public:
    ListaSimple() : primero(nullptr), ultimo(nullptr), largo(0) {}

    ~ListaSimple() {
        Nodo* actual = primero;
        while (actual != nullptr) {
            Nodo* siguiente = actual->siguiente;
            delete actual;
            actual = siguiente;
        }
    }

    size_t tamaño() const { return largo; }
    bool vacia() const { return largo == 0; }

    void insertarAlInicio(int valor);
    void insertarAlFinal(int valor);
    bool buscar(int valor) const;
    void imprimir() const;
};
```

Guardamos un puntero `ultimo` y un contador `largo` para evitar que insertar al final o pedir el tamaño sean operaciones $O(n)$. Es un *trade-off* clásico: usamos un poco más de memoria (dos punteros extra) para ganar tiempo.

Insertar al inicio

```
void ListaSimple::insertarAlInicio(int valor) {
    Nodo* nuevo = new Nodo(valor, primero);
    primero = nuevo;
    if (ultimo == nullptr) {    // la lista estaba vacía
        ultimo = nuevo;
    }
    largo++;
}
```

Insertar al final

```
void ListaSimple::insertarAlFinal(int valor) {
    Nodo* nuevo = new Nodo(valor, nullptr);
    if (ultimo == nullptr) {    // la lista estaba vacía
        primero = nuevo;
    } else {
        ultimo->siguiente = nuevo;
    }
    ultimo = nuevo;
    largo++;
}
```

Pregunta para pensar: ¿por qué insertarAlFinal es $O(1)$ acá, pero sería $O(n)$ si no tuviéramos el puntero `ultimo` ?

Buscar un elemento

La idea es recorrer la lista desde el principio hasta encontrar el valor buscado o llegar al final:

```
BUSCAR(valor):
    actual ← primero
    MIENTRAS actual ≠ nullptr:
        SI actual.dato == valor:
            RETORNAR verdadero
        actual ← actual.siguiente
    RETORNAR falso
```

Ejercicio: implementá esta función en C++.

Eliminar por valor

Para **borrar por valor** hay que recorrer con dos punteros (uno "anterior" y uno "actual"), porque en la lista simple no hay forma de ir hacia atrás una vez que encontramos el nodo:

```
ELIMINAR(valor):
    anterior ← nullptr
    actual ← primero

    // Buscar el nodo con el valor
    MIENTRAS actual ≠ nullptr Y actual.dato ≠ valor:
        anterior ← actual
        actual ← actual.siguiente

    SI actual == nullptr:
        RETORNAR falso // no estaba

    // Desenlazar el nodo encontrado
    SI anterior == nullptr:
        primero ← actual.siguiente // era el primero
    SINO:
        anterior.siguiente ← actual.siguiente

    SI actual == ultimo:
        ultimo ← anterior // era el último

    LIBERAR actual
    largo ← largo - 1
    RETORNAR verdadero
```

Pregunta para pensar: ¿por qué necesitamos el puntero anterior ? ¿Qué pasa si solo tenemos actual ?

Ejercicio: implementá esta función en C++. Prestá atención a los casos borde: eliminar el primero, eliminar el último, eliminar de una lista con un solo elemento.

Recorrer la lista

```
void ListaSimple::imprimir() const {
    Nodo* actual = primero;
    while (actual != nullptr) {
        std::cout << actual->dato << " ";
        actual = actual->siguiente;
    }
    std::cout << std::endl;
}
```

Tamaño de la lista

Como guardamos `largo` como campo de la clase y lo actualizamos en cada inserción/borrado, `tamaño()` es simplemente devolver ese campo: **O(1)**.

Si no lo guardáramos, habría que recorrer toda la lista contando nodos: $O(n)$. Es el mismo *trade-off* que vimos con arreglos dinámicos: pagar un poco de memoria extra para ganar tiempo.

Destructor: liberar todos los nodos

Este es uno de los errores más comunes en C++: si el destructor de la lista no libera **cada nodo** con `delete`, se produce un *memory leak*. No alcanza con que la lista misma se destruya (por ejemplo, al salir del scope o al hacer `delete lista`): sus nodos viven en el heap y nadie los libera automáticamente.

```
~ListaSimple() {
    Nodo* actual = primero;
    while (actual != nullptr) {
        Nodo* siguiente = actual->siguiente; // lo guardamos ANTES de liberar
        delete actual;
        actual = siguiente;
    }
}
```

¿Por qué guardamos `siguiente` antes de hacer `delete actual`? Porque una vez que liberamos la memoria de `actual`, leer `actual->siguiente` es acceso a memoria inválida (comportamiento indefinido). Guardar el puntero antes es obligatorio.

Pregunta para pensar: si el destructor de `ListaSimple` no libera los nodos, ¿el programa se cae inmediatamente? ¿Cómo se detecta este tipo de error en la práctica? (Pensar en herramientas como `valgrind` o *sanitizers*.)

Repaso de complejidades (lista simple)

Operación	Complejidad
constructor	$O(1)$
destructor	$O(n)$
tamaño	$O(1)$
insertarAlInicio	$O(1)$
insertarAlFinal	$O(1)$
buscar	$O(n)$
eliminar (por valor)	$O(n)$
obtener/modificar en posición i	$O(n)$
eliminar el último	$O(n)$

Punto clave: eliminar el último es $O(n)$ porque necesitamos encontrar el *anteúltimo* nodo para que su `siguiente` pase a ser `nullptr`, y en la lista simple no hay forma de ir "hacia atrás".

Lista doblemente enlazada

Invariante: cada nodo conoce al siguiente **y al anterior**.

Estructura del nodo doble

La diferencia con el nodo simple es que ahora cada nodo tiene **dos punteros**: uno al siguiente y otro al anterior.

```
struct NodoDoble {  
    int dato;  
    NodoDoble* siguiente;  
    NodoDoble* anterior;  
};
```

La clase `ListaDoble` tiene la misma estructura que `ListaSimple`: punteros a `primero`, `ultimo` y un contador `largo`.

Insertar al final (lista doble)

```
INSERTAR_AL_FINAL(valor):
    nuevo ← crear NodoDoble(valor)

    SI ultimo == nullptr:
        // Lista vacía: el nuevo es primero y último
        primero ← nuevo
        ultimo ← nuevo
    SINO:
        // Enlazar en ambas direcciones
        nuevo.anterior ← ultimo
        ultimo.siguiiente ← nuevo
        ultimo ← nuevo

    largo ← largo + 1
```

Ejercicio: implementá esta función en C++. Comparala con `insertarAlFinal` de la lista simple.

Eliminar al final (lista doble)

Esta es la operación que justifica tener el puntero `anterior`: en la lista simple era $O(n)$, acá es $O(1)$.

```
ELIMINAR_AL_FINAL():
    SI ultimo == nullptr:
        RETORNAR // lista vacía

    viejo ← ultimo
    ultimo ← ultimo.anterior

    SI ultimo ≠ nullptr:
        ultimo.siguiiente ← nullptr
    SINO:
        primero ← nullptr // quedó vacía

    LIBERAR viejo
    largo ← largo - 1
```

Pregunta para pensar: ¿por qué no necesitamos recorrer toda la lista para encontrar el anteúltimo, como en la lista simple?

Ejercicio: implementá esta función en C++.

El costo de tener el puntero `anterior` es memoria extra por nodo (un puntero más), pero a cambio ganamos poder recorrer en ambas direcciones y hacer `eliminarAlFinal` en $O(1)$.

Repaso de complejidades (lista doble)

Operación	Lista simple	Lista doble
<code>insertarAlInicio</code>	$O(1)$	$O(1)$
<code>insertarAlFinal</code>	$O(1)$	$O(1)$
eliminar el último	$O(n)$	$O(1)$
obtener/modificar en posición i	$O(n)$	$O(n)$
recorrido hacia atrás	✗ imposible	✓ $O(n)$

Pregunta para pensar: si necesitás insertar y eliminar frecuentemente en ambos extremos de la estructura (por ejemplo, para implementar un `deque`), ¿lista simple, lista doble o `std::vector`? ¿Por qué?

Lista circular

Invariante: el último nodo, en lugar de apuntar a `nullptr`, apunta al primero. No tiene un "fin" natural.

Estructura de la lista circular

La lista circular usa los mismos nodos que la lista simple, pero con una diferencia clave: **el último nodo apunta al primero** en vez de a `nullptr`.

Un truco útil: guardamos el puntero a `ultimo` (no a `primero`). Como `ultimo->siguiente` nos da el primero, podemos acceder a ambos extremos en $O(1)$.

```
class ListaCircular {  
private:  
    Nodo* ultimo;    // ultimo->siguiente es el primero  
    size_t largo;  
};
```

Insertar al final (lista circular)

```
INSERTAR_AL_FINAL(valor):  
    nuevo ← crear Nodo(valor)  
  
    SI ultimo == nullptr:  
        // Lista vacía: el nuevo se apunta a sí mismo  
        nuevo.siguiente ← nuevo  
        ultimo ← nuevo  
    SINO:  
        // El nuevo apunta al viejo primero  
        nuevo.siguiente ← ultimo.siguiente  
        // El viejo último apunta al nuevo  
        ultimo.siguiente ← nuevo  
        // Actualizamos quién es el último  
        ultimo ← nuevo  
  
    largo ← largo + 1
```

Ejercicio: implementá esta función en C++.

Recorrer la lista circular

La condición de corte no puede ser `actual != nullptr` (¡nunca hay un nullptr!). Hay que detectar cuándo volvemos al punto de partida:

```

IMPRIMIR():
    SI ultimo == nullptr:
        RETORNAR // lista vacía

    primero ← ultimo.siguiete
    actual ← primero
    HACER:
        IMPRIMIR actual.dato
        actual ← actual.siguiete
    MIENTRAS actual ≠ primero // icorta al volver al inicio!

```

Ejercicio: implementá esta función en C++. Usá un `do-while` para asegurarte de recorrer al menos una vez antes de comparar.

Pregunta para pensar: ¿qué pasa si al recorrer una lista circular usás la condición de corte `actual != nullptr` en vez de `actual != primero` ? ¿Por qué es un error tan común?

Repaso de complejidades (lista circular)

Operación	Complejidad
constructor	$O(1)$
insertarAlInicio	$O(1)^*$
insertarAlFinal	$O(1)^*$
recorrido completo	$O(n)$
buscar	$O(n)$

* Siempre que se guarde el puntero a `ultimo`. Si se guardara el puntero a `primero` en su lugar, ambas pasan a ser $O(n)$ porque habría que recorrer toda la lista para encontrar el último.

Iteradores

¿Qué es un iterador?

Hasta ahora, para recorrer cualquiera de nuestras listas tuvimos que exponer el detalle interno: un puntero `Nodo*` que el código externo mueve con `actual = actual->siguiete`. Esto tiene un problema: **quien recorre la lista necesita conocer su representación interna.**

Un **iterador** es un objeto que abstrae "la posición actual dentro de un recorrido", ofreciendo una interfaz uniforme sin importar cómo está implementada la estructura por dentro:

- `*it` → obtener el elemento actual.
- `++it` → avanzar al siguiente.
- `it != otro` → comparar posiciones (típicamente contra `end()`).

Es la misma idea detrás de un `for` de C++ con rango (`for (auto& x : contenedor)`) y de todos los iteradores de la STL (`std::vector<T>::iterator` , `std::list<T>::iterator` , etc.).

Pregunta para pensar: si mañana cambiamos la implementación interna de `ListaSimple` (por ejemplo, para usar un arreglo en vez de nodos enlazados), ¿el código que hace `for (int x : lista)` tendría que cambiar?

Implementando un iterador propio

Para que una clase soporte el `for` basado en rango de C++, necesitamos:

1. **Una clase `Iterador`** (o `iterator`) que representa "la posición actual" dentro del recorrido
2. **Métodos `begin()` y `end()`** en la clase contenedora que devuelvan iteradores

Métodos que debe tener la clase `Iterador`

El iterador debe definir **tres operadores** para que el `for` funcione:

Operador	Firma	¿Para qué se usa?
<code>operator*</code>	<code>int& operator*()</code>	Desreferenciar: obtener el elemento en la posición actual. Es lo que te da el valor cuando escribís <code>*it</code> o cuando el <code>for</code> asigna <code>x</code> en <code>for (int x : lista)</code> .
<code>operator++</code>	<code>Iterador& operator++()</code>	Avanzar: pasar a la siguiente posición. El <code>for</code> lo llama automáticamente al final de cada iteración. Devuelve una referencia al iterador mismo para permitir encadenar operaciones.

Operador	Firma	¿Para qué se usa?
<code>operator!=</code>	<code>bool operator!=(const Iterador& otro)</code>	Comparar: saber si dos posiciones son distintas. El <code>for</code> lo usa para comparar contra <code>end()</code> y saber cuándo terminar.

Métodos que debe tener la clase contenedora

Método	Firma	¿Qué devuelve?
<code>begin()</code>	<code>Iterador begin() const</code>	Un iterador apuntando al primer elemento . En nuestra lista, sería <code>Iterador(primer)</code> .
<code>end()</code>	<code>Iterador end() const</code>	Un iterador representando "después del último elemento". En nuestra lista, sería <code>Iterador(nullptr)</code> . No apunta a un elemento válido ; solo sirve para comparar.

Estructura del iterador

El iterador es una clase anidada dentro de `ListaSimple` que guarda un puntero al nodo actual:

```
class Iterador {
private:
    Nodo* actual;

public:
    Iterador(Nodo* nodo) : actual(nodo) {}

    // Los tres operadores van acá...
};
```

Los tres operadores que hay que implementar:

```
operator*():  
    RETORNAR actual.dato  
  
operator++():  
    actual ← actual.siguiente  
    RETORNAR este iterador  
  
operator!=(otro):  
    RETORNAR actual ≠ otro.actual
```

Y en la clase `ListaSimple` :

```
begin():  
    RETORNAR Iterador(primerio)  
  
end():  
    RETORNAR Iterador(nullptr)
```

Ejercicio: implementá el iterador completo en C++ y verificá que funcione con `for (int x : lista) .`

Cómo se traduce el for basado en rango

Cuando escribís:

```
for (int x : lista) {  
    std::cout << x << " ";  
}
```

El compilador lo traduce automáticamente a:

```
for (auto it = lista.begin(); it != lista.end(); ++it) {  
    int x = *it;  
    std::cout << x << " ";  
}
```

Fijate cómo se usan los tres operadores del iterador:

- `lista.begin()` → crea el iterador en la posición inicial
- `it != lista.end()` → usa `operator!=` para saber si terminó
- `*it` → usa `operator*` para obtener el elemento actual
- `++it` → usa `operator++` para avanzar al siguiente

Con esto, recorrer nuestra lista se ve exactamente igual que recorrer un `std::vector`:

```
ListaSimple lista;
lista.insertarAlFinal(1);
lista.insertarAlFinal(2);
lista.insertarAlFinal(3);

for (int x : lista) {
    std::cout << x << " ";
}

// Salida: 1 2 3
```

El `for` basado en rango **no es magia**: solo requiere que la clase tenga `begin()` y `end()`, y que el tipo devuelto (el iterador) soporte `*`, `++` y `!=`.

¿Por qué desacoplar el recorrido de la estructura?

El iterador es una capa de abstracción entre **quien recorre** y **cómo está guardada** la información:

- El código que usa `for (auto& x : lista)` no sabe (ni le importa) si por dentro hay nodos enlazados, un arreglo o un árbol.
- Se puede cambiar la implementación interna de la estructura sin romper el código que la recorre, siempre que el iterador mantenga la misma interfaz.
- Permite escribir algoritmos genéricos (buscar, sumar, ordenar) que funcionan sobre **cualquier** contenedor que exponga iteradores — es exactamente lo que hace la STL con `std::find`, `std::sort`, `std::accumulate`, etc.

Pregunta para pensar: ¿por qué `std::sort` funciona sobre `std::vector` pero no compila sobre un iterador de `std::list` (o de nuestra `ListaSimple`)? Pensarlo en términos de qué operaciones necesita `sort` (acceso aleatorio) contra las que ofrece un iterador de lista enlazada (solo avanzar de a uno).

Localidad espacial y cache

¿Por qué importa la localidad espacial?

Cuando el procesador necesita un dato que no está en cache, tiene que traerlo desde la RAM. Este proceso (un *cache miss*) es lento: la RAM es **50-200 veces más lenta** que la cache L1.

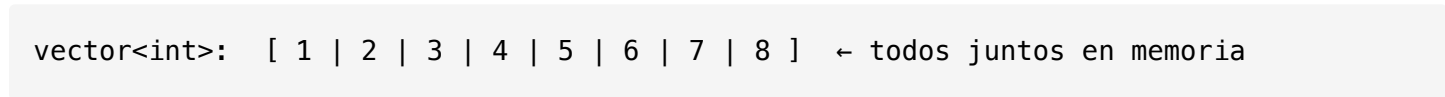
Pero el procesador no trae solo el byte que necesita: trae un **bloque completo** de memoria contigua (típicamente 64 bytes), llamado *línea de cache*. La idea es que si accedés a un dato, probablemente pronto vas a acceder a los datos vecinos.



Localidad espacial significa que los datos que se usan juntos están guardados en posiciones de memoria cercanas. Si tu estructura tiene buena localidad espacial, un solo cache miss te trae muchos datos útiles.

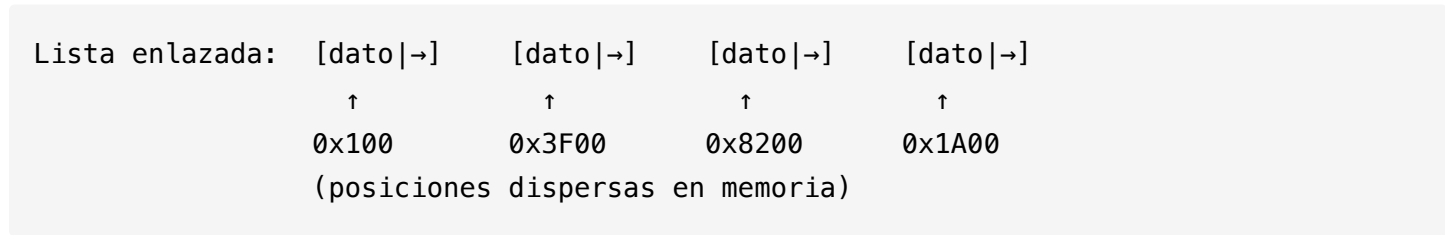
Vector vs lista enlazada: el impacto real

Un `std::vector` guarda todos sus elementos en un bloque contiguo de memoria:



Al recorrer el vector, el procesador trae varios elementos por cada acceso a RAM. En un vector de enteros de 4 bytes, una línea de cache de 64 bytes te trae **16 elementos** de una vez.

Una lista enlazada tiene cada nodo en una posición distinta del heap:



Al recorrer la lista, cada nodo puede causar un cache miss porque están dispersos. El procesador trae un bloque de 64 bytes, pero solo usás 8-16 bytes (el dato + el puntero) antes de saltar a otra posición completamente distinta.

Resultado práctico: aunque recorrer una lista y recorrer un vector son ambos $O(n)$, el vector puede ser **10-50 veces más rápido** en la práctica por aprovechar la cache.

Pregunta para pensar: si la lista enlazada tiene esta desventaja tan grande, ¿por qué sigue siendo útil? ¿En qué casos la usarías igual?

Costo amortizado de `std::vector`

¿Qué pasa cuando el vector se llena?

Un `std::vector` guarda sus elementos en un arreglo dinámico con cierta **capacidad** reservada. Cuando se llena e intentás agregar un elemento más, tiene que:

1. Reservar un arreglo nuevo más grande (típicamente el doble de capacidad)
2. Copiar todos los elementos al arreglo nuevo
3. Liberar el arreglo viejo
4. Agregar el nuevo elemento

Esto es claramente **$O(n)$** cuando ocurre. Pero la clave está en que no ocurre en cada inserción.

Una analogía: la alcancía

Imaginate que tenés una alcancía y cada día metés \$1. Pero cada vez que la alcancía se llena, tenés que:

1. Comprar una alcancía el doble de grande
2. Pasar todas las monedas a la nueva
3. Romper la vieja

Pasar las monedas es "trabajo extra". La pregunta es: **¿cuánto trabajo hacés en promedio por cada moneda que guardás?**

El ejemplo concreto

Supongamos que el vector empieza con capacidad 1:

Inserción 1:	[1]	→ capacidad 1, solo inserto (costo: 1)
Inserción 2:	[1,2]	→ ilлено! creo array de 2, copio 1, inserto (costo: 1+1)
Inserción 3:	[1,2,3,_]	→ ilлено! creo array de 4, copio 2, inserto (costo: 2+2)
Inserción 4:	[1,2,3,4]	→ hay lugar, solo inserto (costo: 1)
Inserción 5:	[1,2,3,4,5,_,_,_]	→ ilлено! creo array de 8, copio 4, inserto (costo: 4+4)
Inserción 6:	hay lugar, solo inserto	(costo: 1)
Inserción 7:	hay lugar, solo inserto	(costo: 1)
Inserción 8:	hay lugar, solo inserto	(costo: 1)

Sumemos los costos

Para insertar 8 elementos gastamos:

- Inserciones simples: $1+1+1+1+1+1+1+1 = 8$
- Copias: $1+2+4 = 7$
- **Total: 15**

Costo promedio por inserción: $15/8 \approx 1.9$

El patrón general

Las copias "caras" ocurren cada vez menos seguido:

- Copio 1 elemento en la inserción 2
- Copio 2 elementos en la inserción 3
- Copio 4 elementos en la inserción 5
- Copio 8 elementos en la inserción 9
- ...

La suma $1+2+4+8+\dots+n/2 = n-1$ (es una serie geométrica).

Entonces para n inserciones:

- Costo de insertar: n
- Costo de copiar: $\sim n$
- **Total: $\sim 2n$**

Dividido por n inserciones = **2 operaciones en promedio = $O(1)$ amortizado**

¿Por qué "amortizado"?

Porque las inserciones "caras" se "pagan" con las baratas. Es como si cada inserción barata guardara un poco de crédito para cuando toque una cara.

Importante: " $O(1)$ amortizado" **no significa que cada operación sea $O(1)$** . La inserción 5 costó 5 operaciones, no 1. Pero si hacés muchas inserciones, el promedio es $O(1)$.

¿Y si agrandamos de a 1 en vez de duplicar?

Ahí sí sería $O(n)$ por inserción:

Inserción 1: copio 0
Inserción 2: copio 1
Inserción 3: copio 2
...
Inserción n: copio n-1

Total de copias: $0+1+2+\dots+(n-1) = n(n-1)/2 = O(n^2)$

Dividido por n inserciones = **$O(n)$ por inserción** → ¡mucho peor!

Por eso **duplicar la capacidad es clave** para que sea $O(1)$ amortizado.

Comparación con arreglo y `std::vector`

Operación	Lista enlazada	<code>std::vector</code>
acceso por índice (<code>v[i]</code>)	$O(n)$	$O(1)$
insertar al inicio	$O(1)$	$O(n)$
insertar al final	$O(1)$	$O(1)$ amortizado
insertar en posición arbitraria (con iterador ya posicionado)	$O(1)$	$O(n)$
localidad de memoria (cache)	mala (nodos dispersos en el heap)	buena (bloque contiguo)
memoria extra por elemento	1 o 2 punteros	ninguna (solo puede sobrar capacidad reservada)

En la práctica, aunque muchas operaciones de la lista enlazada son " $O(1)$ en el papel", `std::vector` suele ser más rápido en recorridos por la localidad de cache: acceder a memoria contigua genera muchos menos *cache misses* que saltar de nodo en nodo por el heap.

Pregunta para pensar: ¿en qué escenario elegirías una lista enlazada por sobre un `std::vector` a pesar de esta desventaja de cache?

Resumen general de complejidades

Operación	Lista simple	Lista doble	Lista circular
insertarAlInicio	$O(1)$	$O(1)$	$O(1)^*$
insertarAlFinal	$O(1)$	$O(1)$	$O(1)^*$
eliminar el primero	$O(1)$	$O(1)$	$O(1)^*$
eliminar el último	$O(n)$	$O(1)$	$O(n)$
buscar / obtener en posición i	$O(n)$	$O(n)$	$O(n)$
recorrido hacia atrás	✗	✓	✗ (salvo circular doble)

* Solo si se guarda el puntero al último nodo.

La STL ya provee `std::list` (lista doblemente enlazada) y `std::forward_list` (lista simplemente enlazada). En la materia las implementamos a mano para entender **cómo funcionan por dentro**, pero en código de producción normalmente usarías las versiones de la biblioteca estándar.